C-CLARKE INTERNATIONAL INSTITUTE OF DIGITAL SCIENCES

Testing Documentation

Test strategy, automated suite coverage, and real defects found and fixed.

Program	Diploma in Advanced AI & Software Engineering
Batch	25/26
Project	Smart Hire — Service Marketplace
Team	Nilan Iddawela, Janaka Eranda, Dasun Anjana, Damith Lasantha, Matheesha Milan
Date	21 July 2026

1. Test Strategy

Testing combines automated unit/integration tests (run on every change, no network or real database) with manual and smoke testing against the real running system (Supabase-backed, exactly as deployed). This mirrors how the team actually worked: automated tests catch regressions immediately; manual and smoke testing catch the things only visible in a live browser session against real data.

LEVEL	TOOL	SCOPE
Backend unit/integration	pytest	24 tests, isolated in-memory SQLite, no real network
Frontend unit	Vitest (Angular test builder)	24 tests across 19 spec files
Smoke test	Custom Python script	One full real user journey against the live backend; creates and deletes its own throwaway accounts
MCP protocol test	Custom Python script (real `mcp` client SDK)	Drives the standalone MCP server exactly as a real MCP client would
Manual / exploratory	Live browser sessions	Every customer, provider, and admin page, cross-role flows, visual regressions

2. Automated Test Suite

2.1 Backend — 24 tests

FILE	COVERS
`test_auth.py` (11)	Registration, login, suspended/deactivated blocking, JWT validation
`test_admin.py` (2)	Category management, service moderation
`test_reviews_payments.py` (4)	Review/payment ownership and validation rules
`test_services.py` (3)	Provider CRUD, favourites, role enforcement
`test_users.py` (4)	Profile access, profile update, public provider response

Run with: `cd backend && source venv/bin/activate && python -m pytest app/tests/ -v`

2.2 Frontend — 24 tests across 19 spec files

Covers guards (auth, role), the auth interceptor and service, login/register forms, customer/provider layouts and profile pages, admin moderation pages, the booking-monitoring and category-management components, the payment-status flow, the star-rating display, and the review submission form.

Run with: `cd frontend && npx ng test`

3. Manual & Smoke Testing

The `qa/` folder documents and drives the parts automation doesn't reach:

- **TEST_PLAN.md** — scope, approach, environments, automated vs. manual split.
- **test_cases.md** — the test case matrix mapping features to specific cases.
- **manual_test_checklist.md** — exploratory checklist for visual and cross-role UX.
- **smoke_test.py** — one full journey (register → browse → book → accept → complete → review → pay) against the real running backend, self-cleaning.
- **test_mcp_server.py** — spawns the standalone MCP server and drives it over the real MCP protocol (list tools, call each one), not just direct function calls.

4. Defects Found and Fixed

Eight real defects were found during development and testing, root-caused, and fixed. Each is logged as a bug report on the project's Jira board under Testing & Quality Assurance.

`BUG-1` **Zoneless change detection: spinners never resolve after HTTP responses**
Several components mutated a plain field inside an `HttpClient.subscribe()` callback. The app has no zone.js and never opts into zoneless mode either, so that mutation never triggered a re-render.
Resolved: Injected `ChangeDetectorRef` and called `.detectChanges()` in every affected subscribe callback.

`BUG-2` **Table action cells with display:flex break row-height and produce jagged dividers**
Three tables applied `display: flex` directly to a <td>, which breaks the table's row-height layout.
Resolved: Moved the flex layout to an inner in all three tables.

`BUG-3` **Admin payments table date column always blank**
The API response was missing `created_at` / `updated_at`, so the frontend never received them.
Resolved: Added both fields to the response schema.

`BUG-4` **Navbar notification bell always shows a fake "2 unread" for every user**
The navbar had its own notification code with a fake list built in, used both as the default and when something went wrong.
Resolved: Deleted the duplicate logic; navbar now uses the existing, correct notification store.

`BUG-5` **"Verified provider" badge hardcoded true on every single service card**
The API response never had a verification field at all — the frontend just always rendered the static text, despite a full admin verification workflow already existing.
Resolved: Added `provider_verified` to the service response, computed from the real verification status.

`BUG-6` **App fails to boot after merge: OPENAI_API_KEY required env var with no default**
A teammate's merge made a previously-optional setting required with no default, crashing the backend for anyone without the key configured.
Resolved: Reverted to optional; the AI service already guarded for a missing key.

`BUG-7` **500 error on /admin/reviews and /admin/dashboard-stats after merge (missing DB column)**
A new column was added to the code but not to the live database. This project doesn't use database migrations — it only creates new tables automatically, it doesn't update existing ones.
Resolved: Added the column directly in the live database and backfilled existing rows.

`BUG-8` **Admin dashboard revenue/bookings chart was 100% hardcoded fake data**
The chart rendered invented numbers with no relationship to real bookings or payments.
Resolved: Added a real revenue-timeseries endpoint and wired the chart to it.

5. Testing Evidence Summary

METRIC	RESULT
Backend automated tests	24 / 24 passing
Frontend automated tests	24 / 24 passing
Defects logged	8
Defects resolved	8 / 8
MCP protocol test	Passing (real client session against the live server)